

● L1

🔄 L2

🤖 ANDROID

🛡️ MASWE-0007

🔍 TEST



MASTG-TEST-0201: Runtime Use of APIs to Access External Storage

Overview

Android apps use a variety of APIs to access the external storage ([📖 External Storage](#)). Collecting a comprehensive list of these APIs can be challenging, especially if an app uses a third-party framework, loads code at runtime, or includes native code. The most effective approach to testing applications that write to device storage is usually dynamic analysis, and specifically method tracing ([🔗 Method Tracing](#)).

Steps

1. Make sure you have [🔗 Frida \(Android\)](#) installed.
2. Install the app.
3. Execute a script to spawn the app with Frida and log all interactions with files.
4. Navigate to the screen of the app that you want to analyse.
5. Close the app to stop Frida.

The Frida script should log all file interactions by hooking into the relevant APIs such as `getExternalStorageDirectory`, `getExternalStoragePublicDirectory`, `getExternalFilesDir` or `FileOutputStream`. You could also use `open` as a catch-all for file interactions. However, this won't catch all file interactions, such as those that use the `MediaStore` API and should be done with additional filtering as it can generate a lot of noise.

Observation

The output should contain a list of files that the app wrote to the external storage during execution and, if possible, the APIs used to write them.

Evaluation

The test case fails if the files found above are not encrypted and leak sensitive data.

To confirm this, you can manually inspect the files using adb shell ([🔗 Host-Device Data Transfer](#)) to retrieve them from the device, and reverse engineer the app ([🔗 Decompile Java Code](#)) and inspect the code ([🔗 Reviewing Decompiled Java Code](#)).

Demos

🤖

MASTG-DEMO-0002: External Storage APIs Tracing with Frida